Instalación previa para el taller Procesamiento de Lenguaje Natural de 0 a RAG

Guía para estudiantes con Windows macOS y Linux

Completa esta preparación antes del taller, con conexión estable a internet. Llegar con Python instalado no es suficiente: también necesitas las bibliotecas, los modelos descargados y las pruebas finales funcionando en tu propia computadora.

Realiza el proceso al menos dos días antes. Reserva varias horas para descargas y pruebas; la duración depende de tu conexión y del equipo. Lleva tu computadora y su cargador. Edición del 5 de noviembre de 2026.

Qué necesitas

Componente	Preparación requerida
Equipo	Sistema de 64 bits. Como objetivo práctico, 16 GB de RAM o más y 30 GB de disco libre; 32 GB ofrecen más margen. Son recomendaciones, no garantías de rendimiento.
Programas	Python 3.12 recomendado o 3.11 existente, Ollama actualizado y navegador. Un editor de texto es útil; **VS Code es opcional pero altamente recomendado**.
Bibliotecas	spacy, sentence-transformers, scikit-learn y umap-learn, con sus dependencias automáticas, incluido PyTorch.
Modelos	es_core_news_sm, BAAI/bge-m3, gemma4:e2b y nomic-embed-text.
Materiales	Solo esta guía para la preparación previa. El docente compartirá los programas de ejercicios y los textos el día del evento.

Una GPU no es obligatoria. La CPU puede ejecutar las actividades, pero BGE M3 y la generación local pueden ser lentos. Con 8 GB de RAM, no des por hecho que podrás completar el flujo; prueba antes y comunica cualquier bloqueo al docente.

Ruta de lectura

Sigue solo la sección de tu sistema en las páginas 2 a 4. Después, todos siguen las páginas 5 a 10. La lista final está en la página 11, las soluciones frecuentes en la 12 y las fuentes oficiales en la 13.

No necesitas cuentas de pago, claves de API, Docker, WSL, Anaconda ni Jupyter para preparar el equipo. No instales herramientas adicionales si ya puedes completar las pruebas.

1 Preparación en Windows

Usa Windows de 64 bits. Ollama solicita Windows 10 22H2 o posterior; conviene un Windows que siga recibiendo actualizaciones. En equipos ARM, confirma primero que los paquetes y Ollama ofrecen soporte para tu arquitectura.

Instalar Python

Abre https://www.python.org/downloads/ e instala el administrador oficial Python install manager. Cierra las terminales anteriores y abre Símbolo del sistema desde Inicio, escribiendo cmd. Los comandos de esta guía para Windows usan cmd, no PowerShell.

```
py install 3.12
py -3.12 --version
```

Resultado esperado: Python 3.12.x. Si ya tienes Python 3.12 y el lanzador clásico py, basta con la segunda línea; no necesitas que exista py install. Si py no se reconoce, revisa la instalación y vuelve a abrir cmd.

Preparar la carpeta y el entorno

Crea una carpeta local para el entorno de trabajo. Los comandos siguientes usan taller_pnl dentro de tu carpeta de usuario; no necesitas recibir archivos del docente para prepararla. Si ya existe, entra en ella y omite mkdir.

```
mkdir "%USERPROFILE%\taller_pnl"
cd /d "%USERPROFILE%\taller_pnl"
py -3.12 -m venv .venv
.venv\Scripts\activate.bat
python --version
python -m pip --version
```

Resultado esperado: el indicador muestra (.venv), Python es 3.12.x y la ruta de pip contiene la carpeta .venv. Si usas una instalación 3.11 ya funcional, crea el entorno con py -3.11 -m venv .venv.

Instalar Ollama

Abre https://ollama.com/download/windows y ejecuta el instalador oficial. Inicia Ollama desde Inicio y déjalo funcionando en segundo plano. Vuelve a abrir cmd si el comando aún no aparece; entra otra vez en la carpeta y activa el entorno.

```
ollama --version
ollama list
```

Resultado esperado: una versión de Ollama y una lista de modelos, que puede estar vacía. Si aparece un error de conexión, inicia la aplicación. Si funciona, continúa en la página 5.

Cada nueva ventana requiere volver a entrar a la carpeta y ejecutar .venv\Scripts\activate.bat. No copies un entorno .venv desde otra computadora.

2 Preparación en macOS

Ollama requiere macOS Sonoma 14 o posterior. En Apple Silicon puede utilizar CPU y GPU; en Mac Intel funciona con CPU y puede ser bastante más lento. Comprueba la versión en el menú Apple y Acerca de esta Mac.

Instalar Python

Abre Terminal. Si ya tienes Python 3.11 o 3.12, comprueba su versión y úsalo para crear el entorno. No elimines ni sustituyas el Python del sistema.

```
python3 --version
```

Si falta una versión adecuada, una ruta es Homebrew. Instálalo siguiendo https://brew.sh y ejecuta las instrucciones finales que muestra para añadir brew a tu terminal. Después, instala Python 3.12:

```
brew install python@3.12
python3.12 --version
```

Si tu macOS o tu Mac Intel no está cubierto por los paquetes de Homebrew, resuélvelo antes con el docente. No dediques la sesión a compilar Python o sus bibliotecas. Conserva una instalación 3.11 o 3.12 que ya supere las pruebas.

Preparar la carpeta y el entorno

Crea una carpeta taller_pnl dentro de tu directorio personal. Aquí guardarás el entorno; los archivos de ejercicios llegarán el día del evento. No necesitas descargarlos para continuar.

```
mkdir -p "$HOME/taller_pnl"
cd "$HOME/taller_pnl"
python3.12 -m venv .venv
source .venv/bin/activate
python --version
python -m pip --version
```

Si usarás un Python 3.11 o 3.12 existente llamado python3, sustituye python3.12 por python3 al crear el entorno. Resultado esperado: (.venv) en el indicador y la ruta de pip dentro de .venv.

Instalar Ollama

Descarga la aplicación desde https://ollama.com/download/mac. Abre el archivo DMG, arrastra Ollama a Aplicaciones y ábrelo. Acepta la configuración del comando de terminal si la solicita. Abre una nueva Terminal si aún no se reconoce.

```
ollama --version
ollama list
```

Resultado esperado: versión y lista de modelos, aunque esté vacía. Vuelve a entrar a la carpeta y activa el entorno si abriste una nueva terminal. Continúa en la página 5.

3 Preparación en Linux

Usa una distribución de 64 bits actualizada. Los comandos de paquetes siguientes corresponden a Ubuntu y Debian. En otras distribuciones, instala Python y venv con el gestor propio; no ejecutes apt si tu sistema no lo utiliza.

Instalar Python y herramientas básicas

```
sudo apt update
sudo apt install python3 python3-venv python3-pip curl
python3 --version
```

Para esta guía utiliza Python 3.11 o 3.12. Debian 12 y Ubuntu 24.04 son ejemplos que incluyen esas ramas. Si tu distribución entrega otra versión, prepara una 3.11 o 3.12 compatible con ella antes del taller; no sustituyas su Python del sistema. No se necesita compilar Python para el taller.

Preparar la carpeta y el entorno

Crea una carpeta taller_pnl dentro de tu directorio personal para guardar el entorno. No necesitas archivos del docente para hacer la preparación previa.

```
mkdir -p "$HOME/taller_pnl"
cd "$HOME/taller_pnl"
python3 -m venv .venv
source .venv/bin/activate
python --version
python -m pip --version
```

Resultado esperado: (.venv), Python 3.11.x o 3.12.x y pip dentro de .venv. Si venv informa que falta ensurepip, instala el paquete venv correspondiente a tu versión de Python y vuelve a crear el entorno.

Instalar e iniciar Ollama

Usa el instalador oficial. El siguiente comando descarga y ejecuta el instalador de Ollama; necesitarás permisos para instalar el servicio.

```
curl -fsSL https://ollama.com/install.sh | sh
ollama --version
ollama list
```

Si el servicio no está activo y tu distribución utiliza systemd:

```
sudo systemctl start ollama
systemctl status ollama --no-pager
```

Resultado esperado: servicio active (running). Si tu sistema no usa systemd, abre otra terminal, ejecuta ollama serve y déjala abierta. Hazlo solo si no hay ya un servicio escuchando. Continúa en la página 5.

No uses sudo pip install ni --break-system-packages. Las bibliotecas del taller se instalan dentro de .venv, con tu usuario.

4 Bibliotecas y modelo de español

A partir de aquí, los comandos sirven en los tres sistemas si tienes el entorno .venv activado y estás en la carpeta taller_pnl que acabas de crear. Copia cada comando completo, aunque Word lo muestre en varias líneas.

Instalar las bibliotecas

```
python -m pip install --upgrade pip
python -m pip install spacy sentence-transformers scikit-learn umap-learn
python -m pip check
```

Resultado esperado de la última línea: No broken requirements found. Se instalarán spaCy, sentence-transformers, scikit-learn y umap-learn, junto con sus dependencias. PyTorch y NumPy llegan como dependencias; no instales el paquete llamado umap ni sklearn por separado.

```
python -c "import spacy, sentence_transformers, sklearn, umap, torch, numpy; print('Bibliotecas disponibles')"
```

La primera importación de UMAP puede tardar. Una advertencia informativa no equivale a un fallo; una excepción o traceback sí requiere revisión.

Descargar el modelo de español

```
python -m spacy download es_core_news_sm
python -m spacy validate

python -c "import spacy; nlp=spacy.load('es_core_news_sm'); doc=nlp('María trabaja en México.'); assert len(doc)>0; assert all(t.pos_ for t in doc); print('spaCy listo')"
```

Resultado esperado: el modelo es compatible y aparece spaCy listo. El paquete spaCy y el modelo de español son descargas distintas: importar spaCy por sí solo no verifica el modelo.

Guardar las versiones que funcionaron

```
python -m pip freeze > versiones_instaladas.txt
```

Conserva este archivo para diagnóstico. El comando de instalación no fija versiones exactas; dos instalaciones en fechas distintas pueden resolver versiones diferentes. Tras superar todas las pruebas, evita actualizar el entorno justo antes del taller.

5 Descargar y probar BGE M3

En el taller se utilizará BAAI/bge-m3 para calcular embeddings. Este modelo se descarga desde Hugging Face y se utiliza mediante sentence-transformers. Es independiente de los modelos guardados por Ollama.

Descargar y ejecutar una prueba pequeña

Con internet y .venv activado, ejecuta:

```
python -c "from sentence_transformers import SentenceTransformer; m=SentenceTransformer('BAAI/bge-m3', device='cpu'); v=m.encode(['Prueba del taller']); assert v.shape==(1,1024); print('BGE M3 listo', v.shape)"
```

Resultado esperado: BGE M3 listo (1, 1024). La primera ejecución descarga varios archivos y pesos grandes. La prueba fuerza CPU para comprobar la ruta más general; los ejercicios del evento pueden elegir aceleración automáticamente.

No interrumpas el proceso mientras esté descargando o cargando el modelo. El tiempo depende del equipo y de la red. Una descarga completa no demuestra por sí sola que el modelo cabe en memoria: por eso se calcula un vector.

Qué conservar

Hugging Face suele guardar los modelos en ~/.cache/huggingface/hub. En Windows, ~ corresponde a la carpeta de tu usuario. La ubicación puede cambiar si configuraste HF_HOME o HF_HUB_CACHE. No borres esa caché antes del taller.

Ejecuta las descargas y las pruebas con el mismo usuario con el que asistirás. Copiar únicamente la carpeta del taller no incluye necesariamente las cachés de modelos. Copiar .venv tampoco prepara otro equipo: hay que recrearlo allí.

Si la descarga falla

Comprueba el espacio libre y vuelve a ejecutar el mismo comando con una conexión estable. Si aparece un error de certificado o acceso de red, prueba una red sin las restricciones institucionales o solicita soporte. No desactives la verificación de certificados.

No necesitas clonar todo el repositorio del modelo ni descargar manualmente sus variantes ONNX o FP16. La carga con SentenceTransformer obtiene los archivos que necesita para esta actividad.

Si falta memoria

Cierra aplicaciones pesadas y evita ejecutar Ollama y BGE M3 al mismo tiempo. Si la prueba termina por falta de memoria, informa al docente antes del taller para acordar trabajo en pareja o resultados precomputados. Cambiar de modelo por tu cuenta altera los ejercicios y los vectores obtenidos.

6 Descargar los modelos de Ollama

Mantén Ollama en ejecución. El servicio debe responder en http://localhost:11434. No necesitas una clave de API para estos ejercicios locales.

Descargar ambos modelos

```
ollama pull gemma4:e2b
ollama pull nomic-embed-text
```

gemma4:e2b genera respuestas; nomic-embed-text representa textos y preguntas como vectores. Espera a que cada descarga termine correctamente. Evita etiquetas distintas o modelos con sufijo cloud.

Comprobar el inventario

```
ollama list
ollama show gemma4:e2b
ollama show nomic-embed-text
```

Resultado esperado: los dos modelos aparecen en la lista y cada comando show presenta información del modelo sin error. nomic-embed-text puede aparecer con la etiqueta :latest. Esto comprueba su presencia; las páginas 7 y 9 incluyen pruebas de ejecución.

Probar la generación real

```
ollama run gemma4:e2b "Responde en español con una frase breve: qué es un texto"
```

Resultado esperado: una respuesta de texto. Su contenido puede variar. La primera carga puede tardar y consumir bastante memoria; esta prueba permite comprobar que el equipo puede ejecutar el modelo, además de almacenarlo.

nomic-embed-text es un modelo de embeddings: no se prueba como un chatbot. Su ejecución se comprueba mediante una petición local en la página 9.

Disco y versiones

Reserva varios GB para gemma4:e2b y espacio adicional para el programa, BGE M3, las bibliotecas y las cachés. El tamaño de una descarga no es la RAM necesaria para ejecutarla. Consulta el tamaño vigente en la ficha del modelo y en ollama list.

Si el manifiesto solicita una versión más reciente de Ollama, actualiza la aplicación desde su sitio oficial y reiníciala. Repite el comando ollama pull correspondiente; no cambies el nombre del modelo para ocultar el error.

7 Comprobar las bibliotecas de análisis

Estas pruebas utilizan datos pequeños incluidos en los propios comandos. No requieren textos del taller ni archivos del docente. Comprueban que las bibliotecas pueden ejecutar operaciones, además de importarse.

Comprobar operaciones numéricas y agrupamiento

```
python -c "import numpy as np; from sklearn.cluster import KMeans;
x=np.array([[0,0],[0,1],[10,10],[10,11]]);
y=KMeans(n_clusters=2,random_state=42,n_init=10).fit_predict(x); assert y[0]==y[1] and y[2]==y[3]
and y[0]!=y[2]; print('Clustering listo')"
```

Resultado esperado: Clustering listo. La prueba agrupa cuatro puntos en dos grupos claramente separados. Las etiquetas numéricas de los grupos pueden intercambiarse sin cambiar el resultado.

Comprobar la proyección con UMAP

```
python -c "import numpy as np; import umap; x=np.random.default_rng(42).normal(size=(20,4));
y=umap.UMAP(n_neighbors=5,random_state=42).fit_transform(x); assert y.shape==(20,2) and
np.isfinite(y).all(); print('UMAP listo', y.shape)"
```

Resultado esperado: UMAP listo (20, 2). Se proyectan veinte puntos de cuatro dimensiones a dos. La primera ejecución puede tardar porque se compilan internamente algunas operaciones; espera a que termine.

Comprobar el procesamiento del español

```
python -c "import spacy; nlp=spacy.load('es_core_news_sm'); doc=nlp('María trabaja en México.');
assert all(t.pos_ for t in doc); print([(t.text,t.pos_) for t in doc]); print('Español listo')"
```

Resultado esperado: una lista de palabras con categorías gramaticales y Español listo. La predicción exacta puede variar según la versión del modelo.

Interpretar los resultados

Si las tres pruebas terminan sin excepciones, las bibliotecas básicas están operativas. Las advertencias sobre ejecución paralela de UMAP no suelen impedir el cálculo. Si aparece un traceback, guarda el mensaje completo para soporte.

Estas pruebas validan la preparación del entorno. Los ejercicios completos, el corpus y la creación de sus resultados se realizarán cuando el docente entregue el material durante el evento.

8 Comprobar la conexión de Python con Ollama

Los ejercicios de RAG se comunicarán con Ollama desde Python. Estas pruebas usan únicamente la biblioteca estándar de Python y textos breves incluidos aquí; no necesitas instalar un paquete Python llamado ollama.

Probar los embeddings locales

Con Ollama activo y .venv activado, copia este comando completo. Envía un texto al modelo nomic-embed-text a través del servicio local.

```
python -c "import json,urllib.request; data=json.dumps({'model':'nomic-embed-text','input':'Prueba
del taller','keep_alive':0}).encode();
r=urllib.request.Request('http://localhost:11434/api/embed',data=data,headers={'Content-
Type':'application/json'}); v=json.load(urllib.request.urlopen(r,timeout=300))['embeddings'];
assert len(v)==1 and len(v[0])>0; print('Embeddings Ollama listos',len(v[0]))"
```

Resultado esperado: Embeddings Ollama listos seguido de la dimensión del vector. keep_alive igual a cero libera el modelo después de la prueba. No hace falta guardar un índice ni disponer de noticias para esta comprobación.

Probar la generación desde Python

```
python -c "import json,urllib.request;
data=json.dumps({'model':'gemma4:e2b','messages':[{'role':'user','content':'Saluda en español en
una frase breve'}],'stream':False,'keep_alive':0}).encode();
r=urllib.request.Request('http://localhost:11434/api/chat',data=data,headers={'Content-
Type':'application/json'});
text=json.load(urllib.request.urlopen(r,timeout=600))['message']['content']; assert text.strip();
print(text)"
```

Resultado esperado: una respuesta de texto sin errores. El contenido puede variar. El comando espera hasta diez minutos; si supera ese tiempo, el equipo todavía requiere revisión antes del evento.

Si la comunicación falla

Connection refused o un error de conexión suele indicar que Ollama no está iniciado. Un error HTTP 404 puede indicar que falta el modelo; compruébalo con ollama list y descarga el nombre exacto. Un tiempo de espera puede deberse a falta de memoria o a un equipo muy lento.

La dirección localhost:11434 corresponde a tu propia computadora. No necesitas una clave de API ni enviar estos textos a un servicio externo. Las peticiones locales también deben funcionar después de desconectar internet.

9 Comprobación sin conexión

Haz esta prueba después de terminar todas las descargas. Reinicia la computadora, abre la carpeta del taller, activa .venv e inicia Ollama. Desconecta el Wi-Fi o la red de internet; no desactives la interfaz de red local ni el servicio de Ollama.

Activar el modo local de Hugging Face

En Windows con cmd:

```
set HF_HUB_OFFLINE=1
```

En macOS o Linux:

```
export HF_HUB_OFFLINE=1
```

Esta variable solo afecta a la terminal actual. Para descargar archivos después, quítala con set HF_HUB_OFFLINE= en cmd o unset HF_HUB_OFFLINE en macOS y Linux.

Repetir las pruebas esenciales

```
python -c "import spacy; nlp=spacy.load('es_core_news_sm'); doc=nlp('María trabaja en México.');
assert all(t.pos_ for t in doc); print('Español local listo')"

python -c "from sentence_transformers import SentenceTransformer; m=SentenceTransformer('BAAI/bge-
m3', device='cpu', local_files_only=True); v=m.encode(['Prueba sin internet']); assert
v.shape==(1,1024); print('BGE local listo')"

ollama list
ollama run gemma4:e2b "Saluda en español en una frase breve"
```

Repite también el comando de embeddings de la página 9, dentro de esta misma terminal y con internet desconectado. Debe aparecer Embeddings Ollama listos y la dimensión del vector. Así compruebas ambos modelos locales, no solo el chatbot.

Resultado esperado: todas las pruebas terminan sin descargar archivos ni producir errores. Las peticiones a localhost:11434 son locales. Vuelve a conectar internet al finalizar y repite cualquier prueba que haya fallado tras resolver la causa.

El día del taller

Abre la carpeta taller_pnl, activa .venv e inicia Ollama. El docente compartirá entonces los programas de ejercicios y los textos. Conserva el entorno ya preparado y no lo reemplaces por uno de otra computadora.

Guarda el material recibido fuera de la carpeta .venv. Si usas un editor, selecciona como intérprete el Python de este entorno. No reinstales bibliotecas ni descargues modelos si ya superaste las pruebas.

10 Lista final para llegar preparado

Marca cada punto solo después de comprobarlo en la computadora que llevarás. Haber ejecutado un instalador no equivale a haber superado las pruebas.

[ ] Python 3.11 o 3.12 funciona dentro de .venv y sé volver a activar el entorno.

[ ] Creé mi carpeta taller_pnl sin necesitar archivos del docente.

[ ] pip check no informa dependencias rotas y las seis importaciones de la página 5 funcionan.

[ ] es_core_news_sm procesa una frase y asigna categorías gramaticales.

[ ] BAAI/bge-m3 produce un vector de 1024 dimensiones sin conexión.

[ ] Las pruebas de agrupamiento y UMAP de la página 8 terminan correctamente.

[ ] Ollama está instalado y puedo iniciarlo después de reiniciar el equipo.

[ ] ollama list muestra gemma4:e2b y nomic-embed-text.

[ ] gemma4:e2b genera texto tanto desde la terminal como desde Python.

[ ] nomic-embed-text genera un vector desde Python con el comando de la página 9.

[ ] Repetí las pruebas de español, BGE M3 y ambos modelos de Ollama sin internet.

[ ] Conservo versiones_instaladas.txt, el entorno y las cachés con el mismo usuario.

[ ] Llevaré cargador y suficiente espacio libre; comuniqué al docente los errores pendientes.

Si algo sigue fallando

Envía al docente antes del taller tu sistema operativo y versión, RAM, versión de Python, versión de Ollama, comando exacto que falló y mensaje de error completo. Incluye versiones_instaladas.txt si el problema es de bibliotecas. No compartas contraseñas ni tokens.

Criterio de preparación completa

El equipo está listo cuando las pruebas terminan correctamente después de reiniciar y sin internet. Si el modelo se descarga pero el equipo no puede ejecutarlo, la preparación sigue incompleta: hace falta acordar una alternativa antes de la sesión.

11 Resolver problemas frecuentes

No se encuentra Python o aparece otra versión

Abre una terminal nueva y repite la activación. En Windows usa cmd y .venv\Scripts\activate.bat; en macOS y Linux, source .venv/bin/activate. Comprueba el ejecutable:

```
python -c "import sys; print(sys.executable); print(sys.version)"
```

La ruta debe apuntar a .venv. Si creaste el entorno con otra versión, conserva el material y recrea solo el entorno con la versión elegida; no borres la carpeta del taller.

PowerShell bloquea la activación

Las instrucciones de Windows están escritas para cmd. Abre Símbolo del sistema y usa activate.bat. No necesitas cambiar la política de ejecución de PowerShell para seguir esta guía.

pip pide compiladores o no encuentra una distribución

Verifica Python 3.11 o 3.12 de 64 bits y actualiza pip dentro del entorno. Lee qué paquete falla. Un problema de arquitectura o versión no se soluciona instalando dependencias al azar. En Mac Intel o Windows ARM puede haber menos paquetes disponibles; solicita soporte antes del taller.

Falta spaCy o su modelo

Si aparece ModuleNotFoundError, comprueba el entorno y repite python -m pip install spacy sentence-transformers scikit-learn umap-learn. Si falta es_core_news_sm, ejecuta python -m spacy download es_core_news_sm con internet y luego python -m spacy validate.

Ollama no conecta o el puerto ya está ocupado

En Windows y Mac inicia la aplicación. En Linux comprueba el servicio. Usa ollama serve solo si no existe ya un servidor activo. Si informa que el puerto 11434 está ocupado, no arranques una segunda instancia: comprueba la existente con ollama list.

Memoria insuficiente o tiempos de espera

Cierra aplicaciones pesadas y ejecuta una actividad a la vez. El equipo puede almacenar un modelo y aun así no tener memoria para cargarlo. Si la consulta supera el tiempo de espera del comando o el proceso se cierra, prueba antes la generación directa con ollama run. Comunica el problema para acordar trabajo en pareja o equipo alternativo.

La prueba sin internet intenta descargar

Revisa que descargaste con el mismo usuario y no eliminaste la caché. Reconecta internet, elimina HF_HUB_OFFLINE de la terminal, repite la descarga y vuelve a hacer la prueba local. Verifica también que el nombre del modelo coincide exactamente.

12 Fuentes y alcance de la guía

Documentación oficial consultada para los comandos de instalación y uso. Los requisitos y la disponibilidad de versiones pueden cambiar; consulta estos enlaces si tu instalador muestra instrucciones distintas.

Python para Windows
https://docs.python.org/3/using/windows.html

Entornos virtuales de Python
https://docs.python.org/3/library/venv.html

Python 3.12 en Homebrew
https://formulae.brew.sh/formula/python@3.12

API de chat de Ollama
https://docs.ollama.com/api/chat

Instalación y modelos de spaCy
https://spacy.io/usage

Instalación de Sentence Transformers
https://www.sbert.net/docs/installation.html

Ficha de BAAI BGE M3
https://huggingface.co/BAAI/bge-m3

API de embeddings de Ollama
https://docs.ollama.com/api/embed

Ollama para Windows
https://docs.ollama.com/windows

Ollama para macOS
https://docs.ollama.com/macos

Ollama para Linux
https://docs.ollama.com/linux

Modelo generador de Ollama
https://ollama.com/library/gemma4:e2b

Modelo de embeddings de Ollama
https://ollama.com/library/nomic-embed-text

Alcance de la preparación previa

Todas las pruebas utilizan comandos incluidos en esta guía y datos de ejemplo. El docente entregará los programas y los textos el día del evento. La preparación verifica las dependencias y la ejecución local de los modelos; los ejercicios completos se realizarán durante el taller.